AI Document Knowledge Platform

System Architecture, Multimodal Ingestion Pipeline & RAG Workflow Specification

Project: Multimodal RAG Platform	**Visual Design:** Midnight Violet + Amber	**Target Stack:** React 18, Spring Boot 3.3, FastAPI
Document Version: 1.0 (Production Specification)	**Vector Dimension:** 384 (all-MiniLM-L6-v2)	**LLM Engine:** Groq (Qwen / Llama 3.3 / GPT-OSS)

1. Executive Summary & Core Capabilities

The **AI Document Knowledge Platform** is an enterprise multimodal Retrieval-Augmented Generation (RAG) system engineered for deep document comprehension across PDFs, Word documents, CSV files, and web pages. It features an observable 7-stage ingestion pipeline with live telemetry, multimodal extraction (tables, images, text), and conversational AI with verified citations.

- **Dual-Engine Multimodal Partitioning:** Extracts high-res text, HTML tables, and embedded raster figures (base64) using Unstructured.
- **Synthetic Multimodal Summarization:** Generates synthetic summaries for extracted charts/tables using Groq vision models before vector indexing.
- **7-Stage Processing Telemetry:** Granular inspector workbench visualizing element discovery, atomic chunking, and vector storage in real time.
- **Strictly Grounded RAG with SSE Streaming:** Real-time token streaming with clickable citation chips referencing exact document IDs, pages, and chunk types.

2. High-Level System Architecture Diagram

```
+-------------------------------------------------------------------------------------------------------------------------------+ | PRESENTATION LAYER (SPA) | | React 18 + Vite + Tailwind CSS (Midnight Violet #0B0A10 & Amber #F5B544 Visual Design Identity) | | - Workspace & Knowledge Panel - Live 7-Stage Pipeline Inspector Modal - Chunk Explorer & SSE Chat Interface |
+-------------------------------------------------------------+-------------------------------------------------------------+ | REST (JSON) & Server-Sent Events (SSE) v Bearer Token / Clerk JWT Authentication
+-------------------------------------------------------------------------------------------------------------------------------+ | APPLICATION BACKEND LAYER | | Spring Boot 3.3 (Java 21) on Port 8081 | | - Project & Document Orchestration - Clerk Security & Auth Context - Local & Cloud File Storage | | - Conversation & Message Persistence - WebFlux WebClient (Reactive Gateway to AI Service) |
+----------------------------+-------------------------------------------------------------+----------------------------+ | JPA / Hibernate | Internal HTTP & SSE Stream v v +-------------------------------------------------------------+
+---------------------------------------------------------+ | DATABASE LAYER | | AI & INGESTION SERVICE | | PostgreSQL 16 + pgvector (:5434) | | FastAPI (:8001) + Python 3.11 / 3.13 | | - Users, Projects, Documents Metadata Tables | | - Unstructured Partitioning (hi_res strategy) | | - Conversations & Messages History | | - Title-Based Semantic Chunking | | - 'document_chunks' table with 384d vector cosine index | | - Groq Multimodal Summarization (Vision & LLM) | | - H2 in-memory profile for autonomous local dev | | - sentence-transformers (384d embeddings) |
+----------------------------+-------------------------+ | - Cohere Reranking API & SSE RAG Answer Generator | ^
+----------------------------+-------------------------+ | Vectors & Chunks | Enqueue Job | v |
+---------------------------------------------------------+ | | MESSAGE BROKER | | | Redis 7 (:6380) | | | - Celery Task Queue Broker | | | - Pipeline Stage Status Cache (TTL 24 Hours) | | +-----------------------------+-------------------------+ | | Consume Tasks | v |
+---------------------------------------------------------+ | | CELERY WORKER | +-------------------------------+ Celery 5.4 Ingestion Pipeline Execution | | - Asynchronous 7-Stage Document Processing Pipeline | | - Automatic Thread Fallback if Redis is Offline |
+---------------------------------------------------------+
```

3. Microservices & Subsystem Specifications

Detailed technical breakdown of each architectural layer, communication protocols, and core responsibilities:

Subsystem	Technology Stack	Port / Protocol	Core Architectural Responsibilities
Frontend SPA	React 18, Vite, TypeScript, Tailwind CSS, Lucide React	Port 3001 (Dev 5173) HTTP/HTTPS	Client-side routing, project workspace, drag-and-drop document upload, real-time SSE token rendering, interactive 7-stage inspector workbench, extracted chunk explorer.
Spring Boot API	Java 21, Spring Boot 3.3, Spring Data JPA, WebFlux	Port 8081 (Dev 8080) REST & SSE	Business domain management, user authentication (Clerk JWT & Dev Bypass), metadata persistence, document storage, and reactive gateway to AI service via WebClient.
FastAPI AI Service	Python 3.11/3.13, FastAPI, Pydantic v2, Uvicorn	Port 8001 (Internal 8000) REST & SSE	Internal AI orchestration exposing endpoints for asynchronous document ingestion triggers, 7-stage status polling, chunk retrieval, vector similarity search, and RAG generation.
Celery Worker	Python Celery 5.4, Unstructured, PyPDF, HuggingFace	Redis Protocol Queue Worker	Distributed execution of the 7-stage document ingestion pipeline. Automatically falls back to internal background threads if the Redis broker is offline.
PostgreSQL + pgvector	PostgreSQL 16, pgvector extension (Cosine Distance)	Port 5434 (Internal 5432) PostgreSQL Wire	Persistent relational store for system entities and high-performance vector store for 384-dimensional document chunk embeddings with indexed cosine similarity search.
Redis Broker	Redis 7 (Alpine)	Port 6380 (Internal 6379) RESP Protocol	Message broker for Celery task queuing and temporary key-value caching of live 7-stage pipeline telemetry metrics with 24-hour expiration.

Visual Design Identity & Design Tokens

The user experience is built on a custom **Midnight Violet + Amber** visual system tailored for deep work:

Token	Hex Value	Visual Role & Implementation
Main Background	#0B0A10	Deep midnight canvas behind main views
Sidebar Background	#100F17	Navigation drawer and project switcher surface
Primary Surface	#171521	Main cards, knowledge panels, chat bubbles
Elevated Surface	#201D2B	Inspector workbench modal, dialogs, card hover state
Primary Accent	#A78BFA (Violet-400)	Active navigation, AI badges, tab highlights, focus rings
Secondary Accent	#F5B544 (Amber-400)	Primary action CTAs, file upload triggers, progress metrics
Subtle Borders	#302B3D	Crisp 1px card dividers and container outlines

4. End-to-End Workflow: The 7-Stage Multimodal Ingestion Pipeline

Document ingestion is fully asynchronous, resilient, and observable. When a file is uploaded or a URL is submitted, the system initiates a 7-stage progression with real-time telemetry tracked in the Document Inspector Modal.

Stage # & Name	Underlying Mechanism	Telemetry & Metrics Captured	Output / Result
Stage 1: Upload to Storage	Spring Boot validates MIME type, hashes content (SHA-256), writes to storage.	Original filename, file size in bytes, MIME type, storage URI.	Document entity created in PostgreSQL with status 'QUEUED'.
Stage 2: Queued	Spring calls /internal/ingest on AI service. Job initialized in Redis/Memory.	Job ID (UUID-12), enqueued timestamp, target Celery worker queue.	Asynchronous Celery task dispatched (or fallback daemon thread).
Stage 3: Partitioning	Unstructured library runs hi_res partitioning on document.	Count of: elements discovered, text sections, tables, images, titles/headers, duration (s).	List of raw atomic document elements with bounding and semantic tags.
Stage 4: Chunking	Title-based chunking grouping related sections under their parent headers.	Atomic elements grouped, total chunks created, average chunk character size, duration (s).	Standardized chunks (max 3000 chars, overlap 500 chars), preserving tables & image refs.
Stage 5: Summarization	Synthetic enrichment: Groq Vision / LLM summarizes extracted tables & figures.	Summarized chunks count, tables summarized, figures/images summarized, duration (s).	Chunks augmented with enhanced_content field for high-recall indexing.
Stage 6: Vectorization & Storage	sentence-transformers computes 384d embeddings. Stored via pgvector.	Vectors stored, embedding dimension (384), pgvector commit time, storage duration.	Indexed entries in document_chunks table with cosine index.
Stage 7: View Chunks	Pipeline finalized. Overall job status marked 'completed'. Inspector unlocks Explorer.	Ready for inspection flag = true, total searchable chunks count.	Document state changed to 'READY'. Available for RAG conversations.

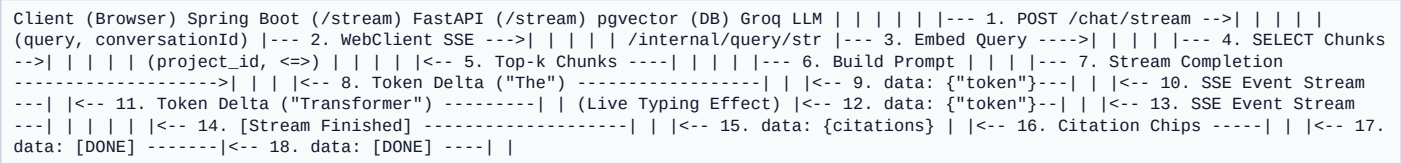
Ingestion Execution Sequence Diagram

5. End-to-End Workflow: Multimodal RAG & SSE Streaming Query

The conversational RAG workflow enforces strict factual grounding to eliminate hallucinations. Answers are streamed token-by-token directly to the client browser over Server-Sent Events (SSE).

- **Step 1: User Query Submission:** The user inputs a query in the ChatView interface, optionally selecting a specific document folder scope ('General' or subfolder). The request is transmitted via HTTP POST to the Spring Boot backend.
- **Step 2: Stream Forwarding via WebClient:** Spring Boot's ConversationService intercepts the query, validates project access, records user message in PostgreSQL, and establishes a reactive Flux SSE stream with FastAPI at /internal/query/stream.
- **Step 3: Query Embedding Generation:** FastAPI's RAG service computes a 384-dimensional dense vector representation of the user query using sentence-transformers/all-MiniLM-L6-v2.
- **Step 4: Vector Similarity Search in pgvector:** The vector store executes an indexed SQL cosine distance query (embedding <=> query_vector) scoped to the active project_id and optional folder_name, retrieving top-k candidates exceeding the similarity threshold.
- **Step 5: Optional Cohere Cross-Encoder Reranking:** If reranking is enabled, candidates are passed to Cohere's rerank-english-v3.0 model to compute fine-grained relevance cross-attention scores, pruning false positives.
- **Step 6: Bounded Prompt Assembly:** FastAPI constructs a secure, bounded system prompt incorporating retrieved text excerpts, formatted HTML tables, and citation markers ([1], [2]). Strict anti-hallucination instructions prohibit answering outside provided evidence.
- **Step 7: Groq LLM Inference & Token Streaming:** The prompt is submitted to the Groq API (Qwen 3.8 / Llama 3.3 / GPT-OSS). Tokens are yielded in real-time as SSE data frames: data: {"token": "..."}\n\n.
- **Step 8: Grounded Citation Delivery:** Upon completion of LLM token generation, FastAPI emits the structured citation metadata array containing chunk ID, document ID, page number, element type, and relevance score, terminated by data: [DONE]\n\n.

Conversational RAG Streaming Sequence Diagram



6. Data Storage & Vector Schema Specification

The system employs a strict relational model for users, projects, and conversations in PostgreSQL, paired with high-performance vector column embeddings via the pgvector extension.

Column Name	Data Type	Constraints / Indexes	Functional Description
id	VARCHAR(64)	PRIMARY KEY	Unique chunk identifier (e.g. chunk_doc123_4).
document_id	VARCHAR(64)	NOT NULL, INDEXED	Foreign reference to parent document entity in Spring Boot.
project_id	VARCHAR(64)	NOT NULL, INDEXED	Multi-tenant workspace isolation boundary.
folder_name	VARCHAR(128)	DEFAULT 'General', INDEXED	Hierarchical folder tag for scoped RAG filtering.
sequence	INT	NOT NULL	Ordinal index of chunk within the source document.
content	TEXT	NOT NULL	Cleaned raw textual representation of the chunk.
enhanced_content	TEXT	NULLABLE	AI-generated synthetic summary for multimodal chunks (tables/images).
page_number	INT	DEFAULT 1	Physical page number in source document for citation grounding.
element_type	VARCHAR(32)	DEFAULT 'text'	Primary element type: text, table, or image.
section_title	TEXT	NULLABLE	Parent section or header discovered during title chunking.
tables_html	JSONB	DEFAULT '[]'	Array of extracted HTML table strings for direct UI rendering.
images_base64	JSONB	DEFAULT '[]'	Array of extracted base64 image strings for preview in Chunk Explorer.
metadata	JSONB	DEFAULT '{}'	Additional file metadata, character count, and coordinate tags.
embedding	vector(384)	pgvector Cosine Index	Normalized 384-dimensional dense semantic embedding.

Relational Entities (Spring Data JPA)

- **UserEntity:** Clerk user ID, email, role, created/updated timestamps.
- **ProjectEntity:** ID (UUID), user ownership, project name, description, document and conversation cascade.
- **DocumentEntity:** ID (UUID), project reference, filename, storage path, MIME type, file size, SHA-256 hash, folder name, status (QUEUED, PROCESSING, READY, FAILED).
- **ConversationEntity:** ID (UUID), project reference, conversation title, message history cascade.
- **MessageEntity:** ID (UUID), conversation reference, sender (USER, ASSISTANT), content, citation JSON, timestamp.

7. Resiliency, Graceful Degradation & Autonomous Local Dev

The platform is engineered to function reliably in both full enterprise containerized clusters and standalone local developer environments with automated degradation fallbacks:

- **Asynchronous Thread Fallback for Celery/Redis:** When Redis or the Celery worker is offline during local development, main.py automatically catches connection exceptions and transparently executes the 7-stage pipeline inside a background Python daemon thread.
- **In-Memory Vector Search Fallback:** If PostgreSQL or pgvector is offline, vector_store.py falls back to a thread-safe in-memory chunk repository using NumPy cosine similarity, enabling complete end-to-end RAG testing without external databases.
- **Multi-Tier Groq Model Fallback:** If the primary configured Groq model encounters rate limits or service unavailability, rag.py executes automated fallback tiers: qwen/qwen3.8-27b → openai/gpt-oss-20b → llama-3.3-70b-versatile → formatted context excerpts.
- **Spring Boot Dev Profile & Clerk Bypass:** A mock development authentication filter permits local testing without a live Clerk internet connection. An in-memory H2 database profile allows running Spring Boot with zero external database configuration.

8. Complete API Routing Matrix

Method & Route	Service	Auth Required	Purpose & Payload
GET /api/v1/projects	Spring Boot	Bearer / Dev	Retrieve list of projects owned by authenticated user.
POST /api/v1/projects	Spring Boot	Bearer / Dev	Create new project (name, description).
POST .../documents	Spring Boot	Bearer / Dev	Upload document (Multipart: file, folder_name).
POST .../documents/url	Spring Boot	Bearer / Dev	Ingest external web page URL into specified folder.
GET .../processing-status	Spring Boot	Bearer / Dev	Poll 7-stage inspector progress and discovery metrics.
GET .../chunks	Spring Boot	Bearer / Dev	Retrieve extracted chunks, HTML tables & base64 images.
POST .../chat/stream	Spring Boot	Bearer / Dev	SSE streaming RAG response with tokens and citation chips.
POST /internal/ingest	FastAPI AI	X-Internal-Secret	Internal trigger for Celery / thread document pipeline.
POST /internal/query/stream	FastAPI AI	X-Internal-Secret	Internal SSE stream endpoint generating grounded answers.

Document Verification: *100% matched against active project code* | ***Format:*** *ISO 32000 PDF Standard*